Project Report

Healthcare Recommendation System

Submitted by: Patil Mrunali
Date: July 2025

Project Overview

The Healthcare Recommendation System is an intelligent, interactive application built using Python and Streamlit. It is designed to assist users in getting health plan recommendations based on their symptoms, with advanced features like chatbot interaction, sentiment analysis, hospital recommendations, and downloadable health reports.

Objectives

- Allow users to receive personalized healthcare recommendations based on their input.
- Integrate seasonal context and demographic data (age, gender) into recommendations.
- Enable chat-based interactions to simulate virtual health assistance.
- Analyze user feedback using sentiment analysis.
- Provide recommendations for hospitals/specialists.
- Enable admin-level insights through a dashboard.
- Export personalized reports as downloadable PDFs.

Technology Stack

Component	Technology
Frontend :	Streamlit
Backend Logic :	Python
Data Analysis :	Pandas
Visualization :	Plotly
NLP / Sentiment Analysis:	TextBlob (uses NLTK internally)
PDF Generation :	ReportLab
Authentication :	Local CSV (basic prototype)
File Formats :	CSV, XLSX

User Authentication

- Registration and login support.
- Role-based access (admin/user).
- Stored user data: username, password, age, gender, role.
- Password hashing was attempted, then skipped for simplicity.
- Admin users can view analytics, others access only their own recommendations.

Symptom-Based Recommendation Logic

- Users enter symptoms (e.g., "fever, headache").
- Symptoms matched with predefined health plans from health_plans.csv.
- Matching uses keyword overlap and seasonal relevance.
- Extra weighting for plans like “hydration” in summer or “immunity” in winter.

Sentiment Analysis

- Feedback entered by the user is analyzed using TextBlob.
- Sentiments categorized as Positive, Neutral, or Negative.
- Helps evaluate effectiveness of the system and user satisfaction.
- Feedback is saved in individual CSV logs (logs_<username>.csv).

Chatbot Mode (MediPal)

- Chatbot interface simulates a virtual health assistant.
- Handles natural interaction like:
  "I have fever and cough."
  "What should I do if I feel tired every day?"
- Remembers symptom history during session.
- Responds with tailored recommendations.
- Friendly tone for users, professional for admins.

Hospital & Specialist Recommendations

- Static database from hospital_list.xlsx.
- If plan = "Cardiology", shows all hospitals related to cardiology.
- Helps users take actionable steps beyond recommendations.

Downloadable Health Report

- A PDF is generated per user session containing:
  - User's name
  - Entered symptoms
  - Recommended health plans
  - Feedback and sentiment

Admin Dashboard

- Aggregated logs of all users.
  - Filter options for:
  - Plan name
  - Sentiment type
- Charts for:
  - Top recommended plans
  - Sentiment distribution
  - Helps monitor user behavior and plan effectiveness.

Project Structure

```
Healthcare_Recommender_System
│
├── app.py                # Main Streamlit app
├── recommendation.py     # Symptom matching logic
├── sentiment.py          # Sentiment analysis
├── logger.py             # Feedback logger
├── data/
│   ├── users.csv
│   ├── health_plans.csv
│   ├── hospital_list.xlsx
│   └── logs_<username>.csv
├── reports/
│   └── report_<username>.pdf
```

Challenges Faced

- Integration of chatbot with recommendation logic.
- Fixing Streamlit reruns and session handling.
- Handling password hashing compatibility.
- Ensuring hospital data remains updated.
- Sentiment misclassification in vague feedback (e.g., "didn't help much").

Final Output

- Working login & registration system.
- Live chatbot with dynamic recommendation.
- Seasonal and demographic-based suggestions.
- Real-time hospital recommendations.
- Sentiment-aware feedback analysis.
- Admin insights and analytics.
- Downloadable reports in PDF format.

Future Scope

- Weather API integration for real-time weather-based recommendations.
- Live hospital availability (via external APIs).
- Multi-language chatbot support.
- Doctor appointment booking module.
- Improved sentiment analysis with BERT or spaCy.

Acknowledgements

Thanks to [Mentor Name] for the guidance and support throughout this project. Grateful for tools like Streamlit, Pandas, TextBlob, and Plotly that made development easier.